



DESARROLLO DE VIDEOJUEGOS

SONIDO, INTERACCIÓN Y REDES

VIDEOJUEGO A VIDEOJUEGO

JUEGOS EN RED

ARQUITECTURA DE INTERNET



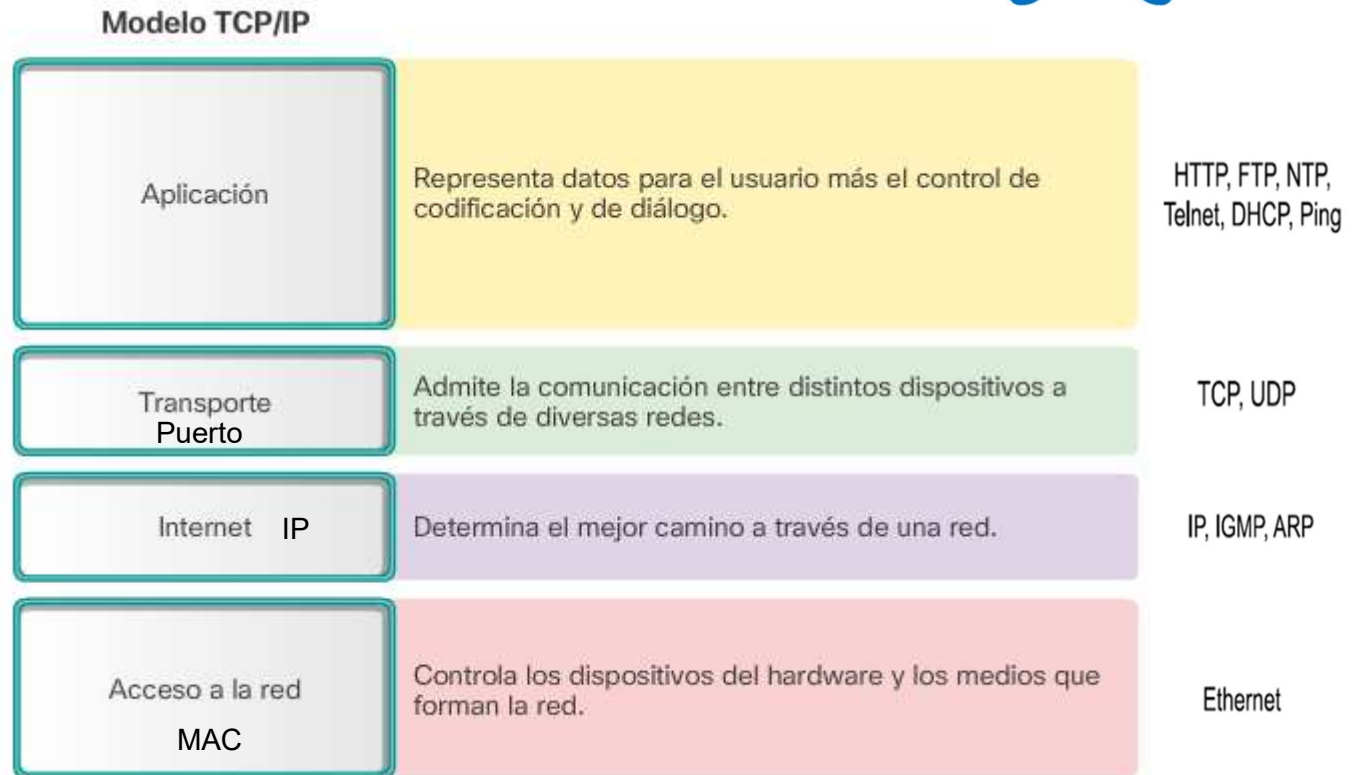
El gran salto cualitativo se produjo con la aparición de Internet como una red de comunicación a escala global.

La comunicación entre dos dispositivos conectados mediante una red se realiza mediante lo que conocemos como un ***protocolo de comunicación***, es decir, un conjunto de normas que definen la lengua en la que hablan dos ordenadores entre sí. El conjunto de protocolos más extendido en la actualidad y el que se utiliza actualmente en Internet es conocido modelo TCP/IP (*Transmission Control Protocol / Internet Protocol*).

CAPAS TCP/IP.-



El modelo TCP/IP está basado en una arquitectura de cinco capas que realizan funciones específicas dentro del proceso de comunicación entre dos aplicaciones.



TIPO DE COMUNICACIÓN



Orientada a la conexión

Requiere el establecimiento de un canal de datos mediante una conexión inicial entre las dos instancias que intervienen en la comunicación.

Las dos instancias negocian entre ellas y crean una conexión dedicada para poder hablar entre sí.

Una vez que se establece el canal, las dos instancias envían y reciben los paquetes de datos a través de este.

En la arquitectura TCP/IP, este tipo de conexión se realiza mediante el uso del protocolo TCP. Ejemplos de protocolos a nivel de aplicación que trabajan con él son HTTP, POP3 o FTP.

TIPO DE COMUNICACIÓN

No orientada a la conexión

Permite enviar paquetes entre dos instancias sin establecer ningún tipo de conexión inicial.

En este caso, el protocolo que se utiliza es el UDP y como ejemplos de aplicaciones basadas en este tipo de conexión tenemos varios sistemas de transferencia de vídeo o audio, DHCP, DNS, etc.



TCP	UDP
Necesita establecer un canal de comunicación.	No necesita de ningún tipo de conexión previa para poder enviar paquetes.
Garantiza que los datos se han transferido íntegramente entre las dos instancias.	No garantiza que los datos se reciban.
Los paquetes se reciben en el mismo orden en que son enviados.	Tampoco garantiza que se reciban en el mismo orden en que se han enviado.
Los paquetes con los datos pueden tener un tamaño de bytes variable.	Los paquetes (en este caso se llaman datagramas) de datos tienen todos la misma longitud.
Si hay algún problema en la transmisión, los paquetes se reenvían, creando más tráfico en la red.	No existe el reenvío de paquetes.
Es un protocolo lento, ya que tiene que garantizar que todos los datos se reciben y que están en orden.	Es un protocolo rápido, ya que no requiere comprobaciones.

TIPO DE COMUNICACIÓN



En el contexto de los videojuegos

La elección del tipo de comunicación es muy importante. Dicha decisión depende de qué tipo de juego queramos implementar

En un juego de estrategia donde cada movimiento es clave para el funcionamiento del sistema, es preferible una conexión TCP que nos garantice que todas las acciones que tome el usuario se ejecuten.

En cambio, en un *First Person Shooter* (FPS), donde se puede actualizar el sistema multitud de veces por segundo, no podemos utilizar un protocolo TCP porque nos podría ralentizar el sistema, así que posiblemente la mejor opción es utilizar UDP y simplemente no preocuparnos por aquellos paquetes que se pierden.

En el mundo de los videojuegos en red existe el R-UDP (*Reliable UDP*). Este protocolo es muy similar a UDP, pero mediante una implementación ligera consigue que tenga la misma fiabilidad que en TCP.

Características de las redes en videojuegos



Los videojuegos son parte de la capa de aplicación, ya que estos se nutrirán de los servicios de todas las capas de la red para poder comunicarse.

Por esto mismo, a los desarrolladores de videojuegos debería importarles, la capa de aplicación y la capa de transporte, debido a que son las únicas a las que pueden llegar a tener acceso.

Para el desarrollo de un modo multijugador en un videojuego, hay que considerar el propio **diseño del videojuego**, ya que hay que tener en cuenta que no todos los juegos requieren de los mismos recursos de red.

Características de las redes en videojuegos



Por su dinamismo. Juego de ajedrez, juego de lucha
Transient (Transitorios) o Persistent (Persistentes)

No se puede permitir que un juego persistente vaya acumulando errores en largas sesiones de juego.

Por tanto, este tipo de clasificación también debe tenerse en cuenta a la hora de tomar decisiones sobre el apartado de comunicaciones de un videojuego multijugador.

Características de las redes en videojuegos



El tráfico de los juegos online no se asemeja a ningún otro y es muy diverso y variable. Es decir: **No hay un estándar.**

El elemento clave para esta diferenciación con los demás tipos de tráfico es la interacción del jugador con el entorno del juego.

Un videojuego no puede permitirse lentitud, ya que el usuario está inmerso dentro de él, y cualquier tipo de retraso le sacaría de la experiencia de juego.

- Posibles fallos de concurrencia.- Es un fallo derivado de la ejecución de varios procesos en paralelo que se comunican entre sí.
- Problemas a la hora de refrescar información.- Mandar la posición de un jugador cada 41 ms podría llegar a saturar la red,

Programación en red



Requiere dos elementos imprescindibles:

- Crear las conexiones entre los clientes, así como saber cómo enviar y gestionar la información.
- Diseñar o incorporar técnicas que nos permitan abstraernos de la comunicación entre las diferentes instancias remotas de un mismo juego.

Es decir, diseñar o utilizar diferentes mecanismos que nos permitan enviar y replicar el estado de nuestro juego y de los elementos que lo componen.



Programación en red

- Herramientas de bajo nivel, como los llamados *sockets*, los cuales nos proporcionan un acceso directo a la capa de red.

En este sentido, tenemos un control total de cada byte de información que se transmite por la red.

- Utilizar un mecanismo de **alto nivel**, que gestione mediante abstracciones todas las conexiones concurrentes y el estado de las mismas.

Programación en red



La herramienta básica para programar en red se basa en el concepto de socket

SOCKETS

Un *socket* es una entidad que permite que un computador intercambie datos con otros computadores. Un *socket* identifica una conexión particular entre dos ordenadores y se compone siempre de cinco elementos:

- Dirección de red (identificador IP),
- Puerto de la máquina origen,
- Dirección de red,
- Un puerto de la máquina destino y,
- El tipo de protocolo (TCP o UDP)

Programación en red



WebSocket

Es un protocolo de comunicación que permite establecer una conexión persistente entre un navegador y un servidor.

La principal diferencia con el HTTP normal es que permite la comunicación bidireccional; el servidor no solo puede responder a las solicitudes de los clientes, sino que también puede enviarle mensajes.

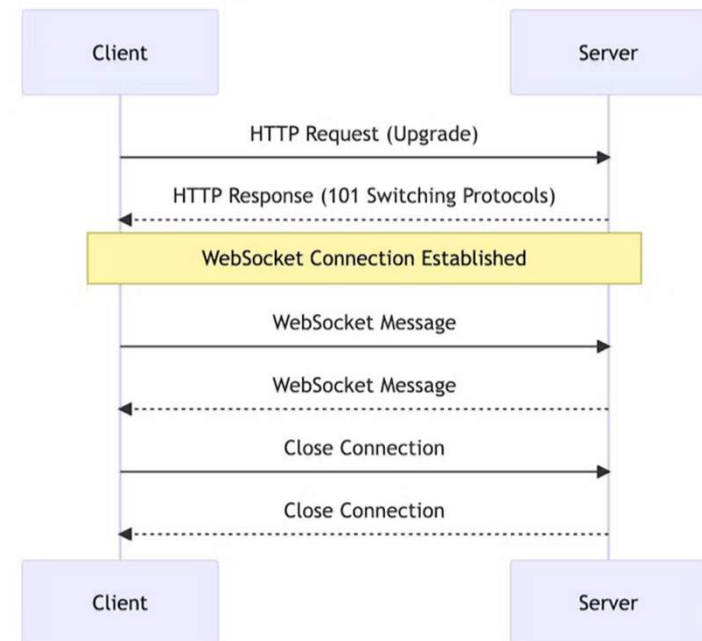
Usa el puerto estándar 80 (HTTP) o 443 (HTTPS), lo que facilita su uso en aplicaciones web que deben funcionar a través de firewalls y proxies.

Programación en red



WebSocket - Como funciona?

1. El cliente envía una solicitud HTTP especial llamada "Solicitud de actualización". Esta solicitud informa al servidor que el cliente desea cambiar a WebSocket.
2. Si el servidor es compatible con WebSocket y acepta cambiar, responde con una respuesta HTTP especial que confirma la actualización a WebSocket.
3. Después de intercambiar estos mensajes, se establece una conexión bidireccional persistente entre el cliente y el servidor. Ambos lados pueden enviar mensajes en cualquier momento, no solo en respuesta a solicitudes del otro lado.
4. Ahora el cliente y el servidor pueden enviar y recibir mensajes en cualquier momento. Cada mensaje de WebSocket está envuelto en "marcos" que indican cuándo comienza y termina el mensaje. Esto permite que el navegador y el servidor interpreten correctamente los mensajes, incluso si se dividen en partes debido a problemas de red.



Programación en red



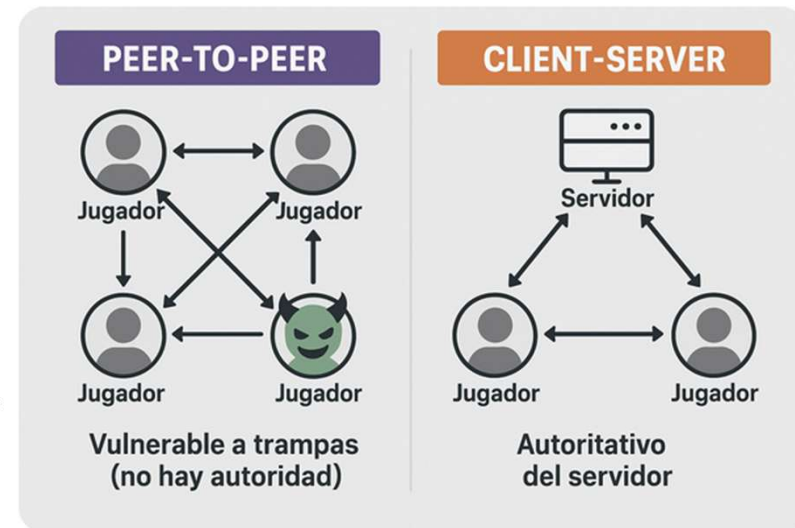
WebSocket - Características

- **Conexión Persistente:** Una vez establecida, la conexión WebSocket permanece abierta, permitiendo a ambas partes enviar datos en cualquier momento.
- **Bajo Overhead:** Comparado con HTTP, tiene un menor overhead en el intercambio de datos después de establecer la conexión inicial.
- **Bidireccional:** Permite enviar y recibir mensajes en ambas direcciones, lo que es más eficiente que las solicitudes HTTP tradicionales que requieren una respuesta del servidor para cada solicitud del cliente.
- <https://github.com/sta/websocket-sharp?ref=hackernoon.com>

Arquitectura de comunicación

La arquitectura de comunicación en un videojuego define cómo los dispositivos (clientes, servidores, peers) se conectan entre sí y cómo intercambian información en tiempo real.

Es un aspecto crítico para determinar el rendimiento, escalabilidad, sincronización y tipo de experiencia de juego (cooperativa, competitiva, persistente).



Arquitectura de comunicación



1. Cliente-Servidor

Esquema:

- Un servidor central procesa toda la lógica del juego.
- Los clientes envían entradas (input) y reciben actualizaciones.

Ventajas:

- Control centralizado (prevención de trampas).
- Persistencia del mundo.
- Escalabilidad (puede usarse con múltiples instancias).

Desventajas:

- Requiere hosting potente.
- Mayor latencia si el servidor está lejos.

Arquitectura de comunicación



2. Peer-to-Peer (P2P)

Esquema:

- No hay servidor central.
- Cada cliente actúa como nodo y comunica directamente con los demás.

Ventajas:

- Fácil de implementar para juegos pequeños.
- Menor latencia en redes locales.

Desventajas:

- Vulnerable a trampas (no hay autoridad).
- Problemas con NAT/firewalls.
- Difícil sincronizar múltiples estados.

Arquitectura de comunicación



3. Cliente-Servidor con replicación distribuida (modelo híbrido)

Esquema:

- El servidor conserva autoridad, pero algunos cálculos se delegan a los clientes (ej. movimiento, físicas).
- Los clientes replican o predicen estados.

Ventajas:

- Reducción de carga en el servidor.
- Mejor experiencia en tiempo real.

Desventajas:

- Requiere mecanismos de reconciliación (rollback, interpolación).
- Posibles inconsistencias si hay pérdida de paquetes.

Diseño de Juegos en Línea Persistentes (Persistent Online Games Design PONG)



Son aquellos en los que el mundo del juego continúa existiendo y evolucionando, independientemente de si un jugador está conectado o no.

Estos juegos tienen una "persistencia" de datos que permite que los personajes, mundos y eventos continúen sin que todos los jugadores tengan que estar activos al mismo tiempo.

Diseño de Juegos en Línea Persistentes (Persistent Online Games Design PONG)



Elementos clave

- **Mundo persistente:** El mundo sigue evolucionando con el tiempo, incluso cuando los jugadores están desconectados.
- **Base de datos centralizada:** Para gestionar la persistencia, es fundamental usar bases de datos que almacenen la información sobre el estado del mundo y las acciones de los jugadores.
- **Eventos sincronizados:** Como cambios en el clima, guerras entre facciones, o misiones globales, deben ser gestionados de forma que los jugadores que regresan a un juego puedan ver cómo ha afectado el tiempo y sus decisiones pasadas al mundo.
- **Sistema de actualización y parches:** En los juegos persistentes, las actualizaciones y los parches pueden cambiar el estado del juego en el servidor de forma que todos los jugadores conectados o desconectados experimenten el mismo entorno.

Diseño de Juegos en Línea Persistentes (Persistent Online Games Design PONG)



Consideraciones en el diseño

- Escalabilidad:** El servidor debe ser capaz de soportar una cantidad masiva de jugadores conectados simultáneamente, con servidores distribuidos si es necesario.
- Seguridad:** La seguridad en los juegos persistentes es esencial, ya que los jugadores invierten tiempo en desarrollar su personaje y pueden perder progresos si la infraestructura no es confiable.
- Economía del juego:** Crear una economía que tenga sentido y no cause desequilibrio entre los jugadores. Esto implica gestionar las recompensas, el comercio y el intercambio de recursos de manera eficiente.

Técnicas Avanzadas para Juegos de Red



Las técnicas avanzadas para juegos de red son métodos y soluciones que permiten mejorar el rendimiento, la estabilidad y la experiencia del usuario en juegos en línea.

Estas incluyen la optimización de la latencia, la gestión de la sincronización de eventos y el manejo de la arquitectura de servidores.

Técnicas Avanzadas para Juegos de Red



Técnicas clave:

- **Interpolación y predicción de movimiento:** Se utilizan para manejar la latencia de la red, haciendo que los movimientos de los jugadores en el juego sean suaves a pesar de las demoras. Se predicen las posiciones de los jugadores mientras esperan la llegada de los datos reales desde el servidor.
- **Compensación de latencia:** Implica técnicas que minimizan los efectos de la latencia en la experiencia de los jugadores.
- **Estado compartido y sincronización de datos:** Los juegos en línea requieren la sincronización de estados entre los clientes y el servidor. La clave es mantener los datos consistentes y evitar problemas como el "rollback" (cuando el estado del juego se retrocede debido a problemas de sincronización).

Técnicas Avanzadas para Juegos de Red



Técnicas clave:

- Desacoplamiento de servidor y cliente:** Los juegos avanzados usan arquitecturas donde el servidor no está completamente acoplado al cliente, lo que reduce la dependencia y mejora la escalabilidad y la seguridad.
- Técnicas de carga y balanceo:** Utilizar varias instancias de servidor y balanceadores de carga para distribuir a los jugadores entre diferentes servidores y asegurar que el rendimiento no se vea afectado por un número excesivo de jugadores en una sola instancia.
- P2P (Peer-to-Peer) vs Servidores dedicados:** Un aspecto avanzado es decidir si el juego usará una arquitectura cliente-servidor tradicional o un modelo de red P2P, en el que los clientes se conectan directamente entre sí, eliminando la necesidad de servidores centrales, pero aumentando los riesgos de piratería y vulnerabilidad.

Herramientas de Comunicación en Juegos de Red



Las herramientas de comunicación en juegos de red son los protocolos y servicios que permiten que los jugadores y el servidor intercambien datos en tiempo real, garantizando que la experiencia del jugador sea fluida y sincronizada.

Tipos de herramientas:

- Sockets y API de redes:** Los sockets permiten la comunicación bidireccional entre el servidor y el cliente, usando TCP o UDP. **TCP** es adecuado para juegos basados en turnos o con poco movimiento en tiempo real. **UDP** es más adecuado para juegos en tiempo real donde el retraso (latencia) debe ser mínimo.
- WebSockets:** Usados principalmente en juegos basados en navegador. Permiten la comunicación persistente entre el cliente y el servidor, lo que permite la actualización en tiempo real sin tener que hacer múltiples solicitudes HTTP.
- Protocolos de sincronización:** Algunas herramientas, como **Photon** o **Mirror (para Unity)**, gestionan la sincronización de eventos entre los jugadores, asegurando que las acciones en el servidor se reflejan correctamente en los clientes.

Herramientas de Comunicación en Juegos de Red



Tipos de herramientas:

- VoIP y chat en vivo:** Los juegos multijugador avanzados ofrecen integración con herramientas de chat de voz en tiempo real (VoIP), como Discord, para facilitar la comunicación verbal entre jugadores.
- Redes distribuidas y CDN (Content Delivery Network):** El uso de redes distribuidas y CDN mejora la entrega de datos a los jugadores, minimizando la latencia, garantizando que los datos del juego se transmitan de manera eficiente y fluida.
- Middleware de red:** Herramientas como **Photon**, **PlayFab**, o **Unity Multiplayer** proporcionan soluciones integradas para la comunicación en juegos multijugador, gestionando las conexiones de red, la sincronización de datos y el mantenimiento de las sesiones de juego.

Herramientas de Comunicación en Juegos de Red



Aspectos clave en las herramientas de comunicación:

- Optimización de ancho de banda:** La transmisión de datos debe ser eficiente para no sobrecargar la red, utilizando compresión y técnicas de "delta compression" para solo enviar cambios pequeños en el estado del juego.
- Seguridad en la comunicación:** Las herramientas de comunicación deben asegurar la protección de los datos del jugador, incluyendo cifrado de la transmisión y autenticación de usuarios.
- Escalabilidad de servidores:** A medida que la base de jugadores crece, las herramientas de comunicación deben ser escalables, con servidores capaces de manejar un volumen creciente de jugadores y solicitudes.